

État d'avancement du projet

Inventaire du réalisé, des écarts et de la trajectoire restante

DATE D'ANALYSE	BRANCHE ANALYSÉE	DERNIER COMMIT	PÉRIMÈTRE	AVANCEMENT GLOBAL
7 septembre 2026	feat/db-and-queries	8a217b7	3 apps · 3 packages	≈ 60 %

1 Résumé exécutif

EgoBot est un monorepo TypeScript (pnpm workspaces + Turborepo) organisé autour d'un pipeline asynchrone : une API Express reçoit un message, l'enfile dans un stream Valkey, un worker le consomme et renvoie des tokens que l'API rediffuse en SSE vers un client Vue 3. En parallèle, un package `logistics-agent` implémente un agent LangChain outillé sur une base PostgreSQL logistique.

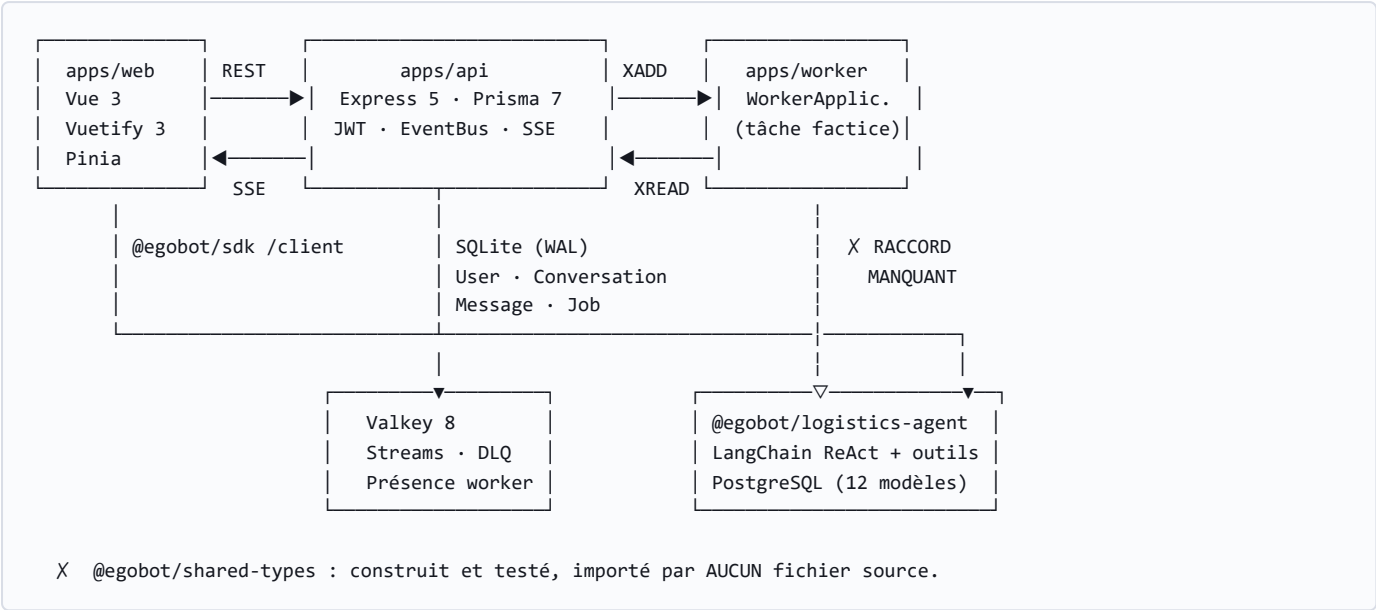
L'infrastructure est mature ; le produit ne l'est pas encore. Le squelette technique — transport temps réel, streams Valkey avec reprise et file de rebut, CI/CD vers GHCR, conteneurisation par environnement — est en place et testé. Le métier, lui, existe mais n'est branché nulle part : le worker répond une phrase codée en dur, et l'agent logistique — la valeur réelle du projet — n'est appelé par aucun composant du pipeline.

96	8	3	0	4
TESTS PACKAGES AU VERT	TESTS WEB EN ÉCHEC	SUITES APPS HORS PÉRIMÈTRE CI	MIGRATION PRISMA VERSIONNÉE	BRANCHES NON FUSIONNÉES

Point de blocage principal

`apps/worker` ne déclare aucune dépendance vers `@egobot/logistics-agent` et n'importe jamais `runLogisticsQuery`. La tâche `CHATBOT` émet une suite de mots figés (« Bonjour ! Je suis un worker d'inférence écrit en Pure TypeScript ! »). Tant que ce raccord n'est pas fait, la chaîne complète n'a jamais été exercée de bout en bout avec de vraies données.

2 Architecture actuelle



MODULE	RÔLE	PILE TECHNIQUE	AVANCEMENT
apps/api	Backend	Express 5, Prisma 7 + better-sqlite3, ioredis, better-sse, JWT, bcrypt, helmet, pino	70 %
apps/web	Frontend	Vue 3, Vuetify 3, Pinia, Vite 5, axios	60 %
apps/workers	Inférence	TypeScript pur sur @egobot/sdk/workers	25 %
packages/sdk	Transport	Client REST/SSE + runtime worker Valkey Streams	90 %
packages/shared-types	Contrats	Schémas Zod 4 (User, Conversation, Job, SSE)	35 %
packages/logistics-agent	Métier	LangChain 1.x, @langchain/openai, Prisma 7 + adapter-pg, Zod	75 %
Intégration bout-en-bout	Produit	Chaîne web → api → worker → agent → PostgreSQL	10 %

Pourcentages estimés à partir de la lecture du code : couverture fonctionnelle vs. périmètre implicite du module.

3 Ce qui est fait

3.1 — Infrastructure & outillage **SOLIDE**

- **Monorepo** pnpm workspaces (`apps/*` , `packages/*`) + Turborepo 2 avec graphe de dépendances, cache et `globalEnv` déclaré.
- **Conteneurisation par environnement** : `Dockerfile.dev` / `.test` / `.prod` pour l'API et le web, `.dev` / `.prod` pour le worker ; 4 fichiers `Compose` (base, dev, test, prod), image web servie par Nginx Alpine.
- **CI/CD GitHub Actions** : job `lint-test-build` puis publication multi-app (matrice api/web/worker) vers GHCR, avec stratégie de tags distincte `main` (latest + version) et `dev` (dev + SHA court).
- **Changesets** initialisé pour le versionnement SemVer par package.
- **Documentation** : 7 `README.md` (racine + un par app/package), diagramme Mermaid d'architecture, guides de mise à jour et de release.

3.2 — `packages/sdk` — transport et runtime worker **MATURE**

- **Client REST** (`/client`) : authentification, conversations, messages, administration.
- **Client SSE** : `connectJobStream` avec callbacks `onToken` / `onStatus` / `onError` , gestion de `Last-Event-ID` et fonction de nettoyage.
- **Runtime worker** (`/worker` , 262 lignes) : consommation de Valkey Streams par groupes, **récupération des messages orphelins via `XAUTOCLAIM`** , **file de rebut (DLQ) après 3 échecs**, publication de présence, boucle de polling optimisée (commit `fedae2` , suppression des boucles d'attente active CPU).
- **Contexte d'exécution** : `ctx.sendToken()` et `ctx.checkCancellation()` pour l'émission incrémentale et l'interruption coopérative.
- 26 tests unitaires, tous au vert.

3.3 — `apps/api` — backend Express **FONCTIONNEL**

- **Authentification** : `register` / `login` / `me` , hachage bcrypt, JWT, middleware `authMiddleware` + `requireAdmin` , et un `sseAuthMiddleware` dédié (le token transite par la query string, `EventSource` n'acceptant pas d'en-tête).
- **Persistance** : Prisma 7 sur SQLite en mode WAL via `@prisma/adapters-better-sqlite3` ; modèles `User` , `Conversation` , `Message` , `Job` avec énumérations mappées et suppression en cascade.
- **EventBus interne** (97 lignes) : publication/souscription par sujet, `subscribeAll` , et un mode requête/réponse corrélé avec délai d'expiration.
- **Diffusion SSE** via `better-sse` : sessions indexées par job et par conversation, et **rejeu des événements manqués** depuis le stream Valkey à partir de `Last-Event-ID` .
- **JobService** : création de conversation implicite, insertion des messages utilisateur/assistant, enfilage `XADD` , agrégation des tokens et consolidation du message final avec ses statistiques (`generated_tokens` , `tokens_per_second` , `time_to_first_token`).
- **TacheService** : tâche de fond remplaçant en `FAILED` les jobs inactifs depuis plus de 15 minutes.
- **Backoffice** : CRUD utilisateurs complet, y compris réinitialisation de mot de passe avec génération aléatoire (`randomBytes`), plus deux flux SSE d'administration (inspection d'une conversation, et debug live de l'EventBus *restreint au développement*, commit `d970ea7`).
- **Durcissement** : `helmet` , CORS configurable, arrêt gracieux fermant les 3 connexions Valkey et Prisma sur `SIGTERM` / `SIGINT` .

3.4 — `apps/web` — interface Vue 3 **À CONSOLIDER**

- Écran d'authentification à onglets (connexion / inscription avec choix de rôle), gestion des erreurs.

- Interface de chat : liste des conversations, création, suppression, **affichage token par token** via SSE.
- **Repli automatique** : sans aucun token reçu après 6 s, le store bascule sur une récupération HTTP de la conversation — le service reste utilisable si le SSE échoue.
- Vue backoffice : gestion des utilisateurs, thème Vuetify clair/sombre suivant le système (commit `9c825b6`).
- 3 stores Pinia (`auth` , `chat` , `backoffice`) et 38 tests unitaires (jsdom + `@vue/test-utils`).

3.5 — `packages/logistics-agent` — cœur métier **AVANCÉ, ISOLÉ**

- **Modèle de données PostgreSQL** : 12 modèles et 5 énumérations couvrant le cycle logistique complet — `Customer` , `Address` , `Order` , `OrderLine` , `OrderLinePrice` , `Delivery` , `DeliveryLine` , `Supplier` , `SupplierProduct` , `Product` , `StockItem` , `StockMovement` . Conception soignée : montants en `Decimal(12,2)` , prix figés par ligne pour préserver l'historique, adresses verrouillables, livraisons partielles, index métier.
- **Générateur de données** (834 lignes) : 8 fournisseurs, 80 articles, 50 clients, 150 commandes ; graine fixe donc reproductible, calculs monétaires **entièrement en centimes entiers** (aucun flottant), et **relecture finale contrôlant 6 invariants** (cohérence des totaux, stock physique vs. logique, quantités expédiées $\leq$ commandées).
- **Services de requête** : commandes, livraisons, inventaire, résolution d'identité client — tous filtrés par `customerId` .
- **Outils LangChain** : `get_order_status` , `get_order_details` , suivi de livraison, disponibilité article ; entrées validées par des schémas Zod documentés.
- **Agent ReAct** : invite système en français interdisant l'invention de données, le SQL arbitraire et la fuite d'informations fournisseur ; garde-fous `modelCallLimitMiddleware` (6 appels) et `toolCallLimitMiddleware` (8 appels).
- **Runtime** `runLogisticsQuery` (221 lignes) : point d'entrée neutre pour un appelant externe, avec diffusion par fragments (`onToken`), annulation coopérative (`shouldCancel`), injection de modèle pour les tests, et repli sur `invoke` si `streamEvents` est absent. **L'agent est reconstruit à chaque appel** — les outils capturent le `customerId` par fermeture, ce qui interdit structurellement la fuite entre clients.

4 Ce qui manque

4.1 — Bloquants

SUJET	CONSTAT ET CONSÉQUENCE
Agent non branché CRITIQUE	<code>apps/worker/package.json</code> ne dépend que de <code>@egobot/sdk</code> et <code>@egobot/shared-types</code> . La tâche <code>CHATBOT</code> émet 12 mots codés en dur avec 120 ms de pause. <code>runLogisticsQuery</code> n'est appelé que par le script manuel <code>scripts/try-agent.ts</code> . <i>Le produit n'existe pas encore comme chaîne fonctionnelle.</i>
Aucune migration Prisma CRITIQUE	Ni <code>apps/api/prisma/migrations/</code> ni <code>packages/logistics-agent/prisma/migrations/</code> n'existent. Seuls les <code>schema.prisma</code> et des fichiers <code>.db</code> sont présents. Aucun déploiement reproductible n'est possible et l'historique du schéma est perdu.
Deux bases sans pont CRITIQUE	L'API stocke <code>User</code> en SQLite ; l'agent interroge <code>Customer</code> en PostgreSQL. Aucune correspondance entre les deux. <code>runLogisticsQuery</code> exige une <code>CustomerIdentityInput</code> qu'aucun composant ne sait produire à partir d'un utilisateur authentifié.
PostgreSQL absent de l'infra CRITIQUE	Les 4 fichiers <code>Compose</code> ne déclarent que <code>valkey</code> (+ <code>api/web/worker</code>). Aucun service <code>postgres</code> , aucune <code>DATABASE_URL</code> dans <code>docker-compose.prod.yml</code> , aucun PostgreSQL dans la CI. La base logistique n'existe qu'en local sur un poste de développeur.
<code>shared-types</code> inutilisé MAJEUR	Le package est déclaré en dépendance par l'API, le worker et le SDK, mais aucun fichier source ne l'importe . Les contrats Zod sont écrits et testés à ~99 % — et n'ont aucun effet. Les DTO circulent en <code>any</code> non validé.

4.2 — Qualité & validation

SUJET	CONSTAT
Tests web en échec	8 échecs sur 38 — <code>chat.spec.ts</code> (3 : nettoyage de flux, erreur de streaming, repli après inactivité), <code>vuetify.spec.ts</code> (2 : thèmes clair/sombre, thème par défaut), <code>App.spec.ts</code> (3 : flux debug EventBus, création et suppression d'utilisateur). Les specs n'ont pas suivi les commits <code>9c825b6</code> et <code>fc51e2</code> (délai de repli passé de 6 s à 20 s sur une autre branche).
Tests API non exécutables	La suite <code>apps/api</code> ne produit aucune sortie et ne se termine pas après 5 minutes : <code>config/valkey.ts</code> ouvre 3 connexions ioredis dès l'import, sans Valkey actif ni délai d'abandon. Aucun test API ne peut donc être vérifié en l'état.
Tests des apps hors CI	<code>vitest.config.ts</code> racine cible <code>include: ["packages/**"]</code> , et le script racine est <code>vitest run</code> (et non <code>turbo run test</code>). L'étape CI <code>pnpm test</code> n'exécute jamais les suites de <code>apps/api</code> , <code>apps/web</code> ni <code>apps/worker</code> — d'où des échecs invisibles au vert.
Étape de lint vide	<code>pnpm lint</code> délègue à <code>turbo run lint</code> , mais aucun des 6 packages ne définit de script lint . Ni ESLint ni Prettier ne sont configurés dans le dépôt. L'étape CI passe systématiquement sans rien vérifier.
Test e2e orphelin	<code>apps/api/src/__tests__/e2e-pipeline.test.ts</code> existe mais n'est pas importé par <code>run-all-tests.ts</code> : il n'est jamais lancé.
Aucun test d'intégration réel	Rien ne valide la chaîne complète avec un vrai Valkey et une vraie base (ni Testcontainers, ni service CI).

4.3 — Code écrit mais non raccordé

Trois composants sont implémentés et couverts par des tests, mais ne sont référencés par aucun chemin d'exécution :

- `middlewares/rateLimit.middleware.ts` — `authRateLimiter` et `messageRateLimiter` ne sont montés sur aucune route de `app.ts`. **Aucune limitation de débit n'est active**, y compris sur `/auth/login`.
- `middlewares/validate.middleware.ts` — `validateRequest` n'est appliqué nulle part. Les corps de requête sont lus directement (`req.body.email`, `req.body.password`) : **aucune validation Zod en entrée d'API**.
- `repositories/worker.repository.ts` — la présence et le décompte des workers ne sont exposés par aucun service ni aucune route. Le backoffice ne peut pas afficher l'état de la flotte.

4.4 — Sécurité & exploitation

SUJET	CONSTAT
Secrets par défaut	<code>JWT_SECRET</code> a pour valeur de repli <code>"super_secret_jwt_key_12345"</code> dans <code>config/env.ts</code> , et <code>docker-compose.prod.yml</code> fixe en clair <code>change_me_in_production_jwt_secret_98765</code> . Un déploiement sans surcharge démarre avec un secret public.
Base de production versionnée	<code>apps/api/prisma/prod.db</code> , <code>prod.db-shm</code> et <code>prod.db-wal</code> sont suivis par Git malgré les règles <code>*.db</code> du <code>.gitignore</code> (ajoutés avant celles-ci). À retirer de l'index.
CORS permissif	<code>CORS_ORIGIN</code> vaut <code>"*"</code> par défaut, y compris en production.
Élévation de privilège	La route publique <code>/auth/register</code> accepte un champ <code>role</code> depuis le client, et le formulaire d'inscription web propose explicitement le choix du rôle. N'importe qui peut se créer un compte administrateur.
<code>KEYS</code> en boucle serrée	<code>JobService.pollSseStreams</code> exécute <code>valkeyStream.keys("jobs:sse:dev:*")</code> toutes les 200 ms. <code>KEYS</code> est une commande O(N) bloquante, déconseillée en production : à remplacer par <code>SCAN</code> ou, mieux, par une souscription événementielle.
Environnement figé à <code>dev</code>	Les clés de stream sont construites en dur : <code>jobs:queue:dev:\${model}</code> et <code>jobs:sse:dev:*</code> côté API, tandis que <code>SseService</code> lit <code>process.env.NODE_ENV</code> . En production, l'API écrit et lit sur <code>dev</code> mais rejoue depuis <code>production</code> — incohérence de préfixe qui casse la reprise SSE .
Pas de sonde de santé	Aucune route <code>/health</code> ou <code>/ready</code> , alors que les images sont déployées via Compose et GHCR. Aucun <code>healthcheck</code> dans les fichiers Compose.
Observabilité limitée	<code>pino</code> est en place, mais aucune métrique, aucun traçage, aucune corrélation de bout en bout au-delà de l'EventBus interne.

4.5 — Fonctionnalités absentes

- **Annulation de job** : `ctx.checkCancellation()` est implémenté et testé côté SDK, mais aucune route API ne permet de déclencher une annulation. Le mécanisme est inatteignable depuis l'interface.
- **Cycle de vie des jobs incomplet** : `JobService` ne traite que la transition vers `COMPLETED`. Les statuts `IN_PROGRESS`, `FAILED` et `CANCELLED` ne sont jamais écrits depuis les événements du worker — seule la tâche de fond « jobs bloqués » produit un `FAILED`, au bout de 15 minutes.
- **Modèle ANALYTICS** : déclaré par le worker mais aucune tâche n'y est enregistrée. Un job soumis sur ce modèle reste sans réponse.
- **Structure frontend** : `App.vue` concentre 571 lignes (authentification, chat, backoffice). Aucun routeur, aucun composant, aucune vue séparée. La maintenance et les tests s'en trouvent durablement compliqués.
- **Pagination et recherche** : absentes des listes de conversations, de messages et d'utilisateurs.
- **Suppression logique inexploitée** : `Conversation.deleted_at` existe au schéma mais aucune logique de restauration ni de purge ne s'appuie dessus.

4.6 — Gestion du dépôt

- **4 branches divergentes non fusionnées** — `dev`, `feat/db-and-queries` (courante), `feat/seed-logistique`, `origin/feat/workers_alexis`. Ces branches touchent des zones proches ; le risque de conflit croît avec le temps.
- La branche courante **ne contient pas** `d67838f` (vue backoffice des conversations + inspection SSE) ni `fc51e2` (rendu Markdown GFM, repli SSE à 20 s), présents sur `dev` et `feat/seed-logistique`.
- `origin/feat/workers_alexis` ajoute d'autres outils à l'agent logistique (`fb193b1`) — travail parallèle à réconcilier avec celui de la branche courante.
- **Changesets** est configuré mais `.changeset/` ne contient que `config.json` : aucun changeset créé, les 6 packages restent figés en `1.0.0`.
- **README racine obsolète** : liens `file:///home/tboutin/...` pointant vers un poste local, et références au nom de paquet historique `@my-11m/*` alors que le renommage en `@egobot/*` est effectif depuis `afb3512`.
- `packages/logistics-agent/.env.example` ne documente que `DATABASE_URL`, alors que l'agent requiert aussi `OPENAI_API_KEY` et accepte `LOGISTICS_MODEL`.

5 Trajectoire recommandée

Ordre proposé : rétablir d'abord un signal de qualité fiable, puis fermer la chaîne fonctionnelle, puis durcir. Chercher à durcir avant d'avoir un pipeline qui fonctionne reviendrait à sécuriser une chaîne qui n'a jamais tourné.

ÉTAPE	ACTION	POURQUOI MAINTENANT
1	Faire tourner les tests des apps en CI : passer le script racine à <code>turbo run test</code> (ou étendre le <code>include</code> de vite) et corriger les 8 échecs web.	Tant que la CI est verte à tort, chaque correction ultérieure repose sur un signal faux.
2	Rendre la suite API exécutable : injecter les clients Valkey plutôt que les créer à l'import, ou fournir un service Valkey en CI.	Un tiers du backend n'est aujourd'hui vérifiable par personne.
3	Ajouter PostgreSQL aux fichiers Compose et à la CI, puis initialiser les migrations Prisma des deux bases (<code>prisma migrate dev --name init</code>).	Prérequis matériel de toute la suite : sans base déployable, rien de métier n'est intégrable.
4	Définir la liaison <code>User</code> (SQLite) ↔ <code>Customer</code> (PostgreSQL) — au minimum une correspondance par e-mail, idéalement un <code>customer_number</code> porté par l'utilisateur.	C'est la pièce manquante qui empêche d'appeler l'agent avec une identité authentifiée.
5	Brancher <code>runLogisticsQuery</code> dans la tâche <code>CHATBOT</code> du worker : <code>ctx.sendToken</code> en <code>onToken</code> , <code>ctx.checkCancellation</code> en <code>shouldCancel</code> . L'interface du runtime a déjà été conçue pour cela.	Premier moment où le produit existe réellement de bout en bout.
6	Écrire un test d'intégration de la chaîne complète (Valkey + PostgreSQL réels, modèle simulé via l'option <code>model</code> déjà prévue par le runtime).	Verrouille l'acquis de l'étape 5 contre toute régression.
7	Corriger le préfixe d'environnement des clés Valkey, câbler <code>validateRequest</code> et les limiteurs de débit, retirer <code>role</code> de <code>/auth/register</code> , faire échouer le démarrage en production sans <code>JWT_SECRET</code> explicite.	Quatre correctifs courts pour des risques disproportionnés — dont une élévation de privilège ouverte.
8	Compléter le cycle de vie des jobs (<code>IN_PROGRESS</code> , <code>FAILED</code> , <code>CANCELLED</code>) et exposer une route d'annulation.	Le SDK sait déjà annuler ; il ne manque que le déclencheur côté API.
9	Imposer <code>@egobot/shared-types</code> aux frontières (contrôleurs API, payloads worker, SDK) et remplacer les <code>any</code> restants.	Le package est prêt et testé : le raccorder est un gain net, sans coût de conception.
10	Fusionner les 4 branches vers <code>dev</code> , puis découper <code>App.vue</code> en vues et composants avec <code>vue-router</code> .	Plus la divergence dure, plus la réconciliation coûte cher.

ÉTAPE	ACTION	POURQUOI MAINTENANT
11	Ajouter ESLint + Prettier, une sonde <code>/health</code> , des <code>healthcheck</code> Compose, et remplacer <code>KEYS</code> par <code>SCAN</code> . Retirer <code>prod.db*</code> de l'index Git et actualiser le README racine.	Dette d'exploitation et d'hygiène, sans urgence fonctionnelle.

Lecture d'ensemble

Le projet ne souffre pas d'un déficit de qualité de code — les parties écrites sont soignées : le générateur de données auto-vérifié, l'isolation par fermeture du `customerId`, la récupération des messages orphelins et la file de rebut témoignent d'un vrai niveau d'exigence. Ce qui manque est du **raccordement** : plusieurs composants aboutis (agent logistique, contrats Zod, limiteurs, validation, dépôt worker) ne sont appelés par personne. Les étapes 1 à 5 ci-dessus ne créent presque aucun code nouveau — elles connectent ce qui existe déjà.